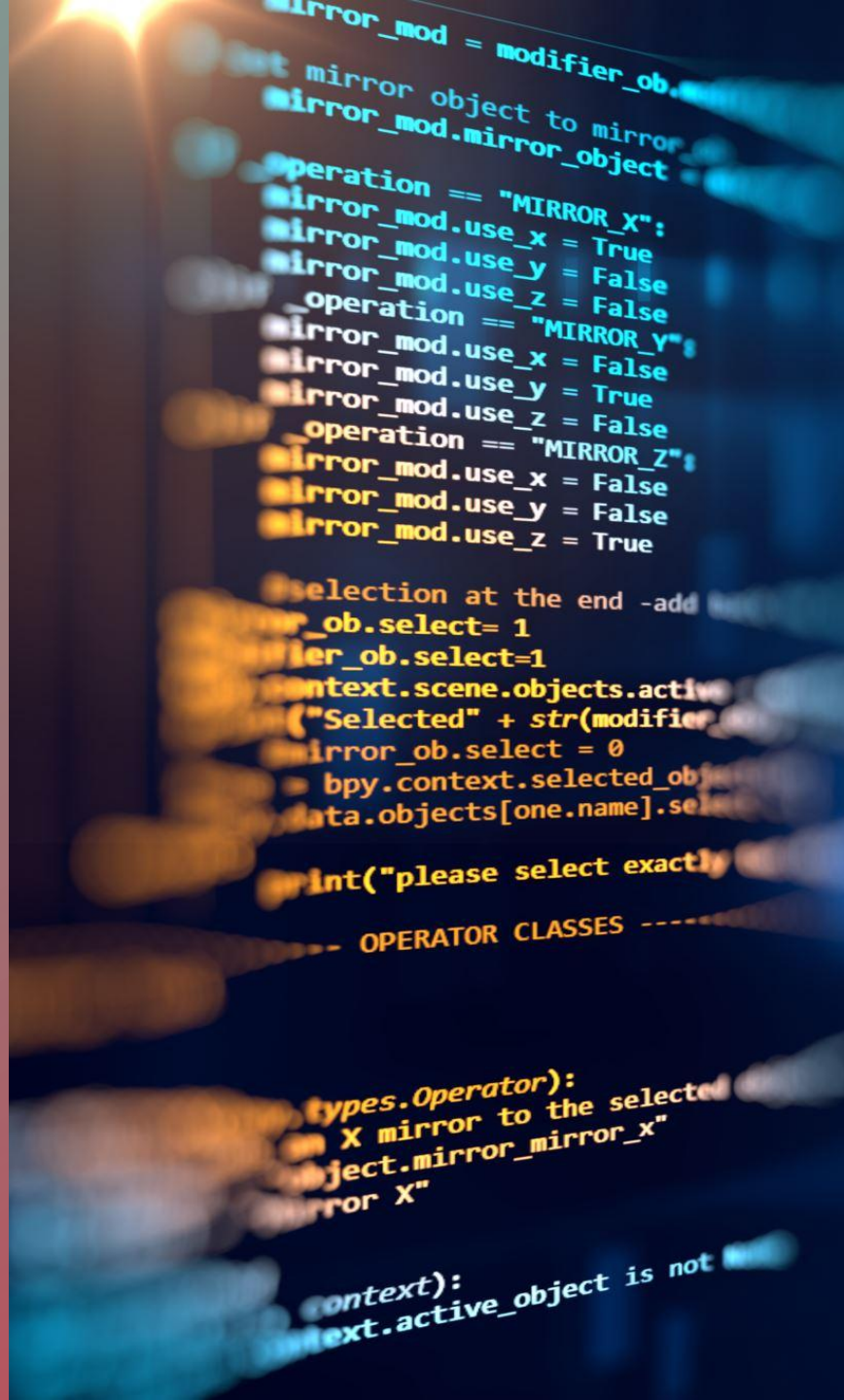
INTRODUÇÃO A ALGORITMOS E ESTRUTURAS DE DADOS



ALGORITMO

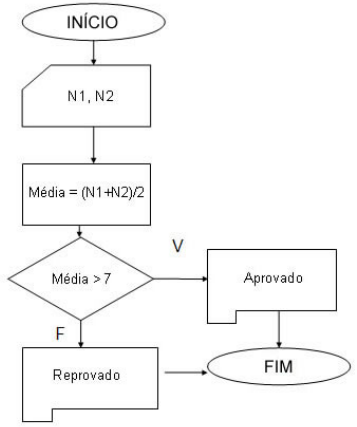
- “Uma sequência de instruções (passos) não ambíguas para resolver um problema, isto é, para obter uma saída requerida para uma entrada legítima, em um período de tempo finito.”

(Levitin, 2012)

ALGORITMO

Problema → Algoritmo → Computador

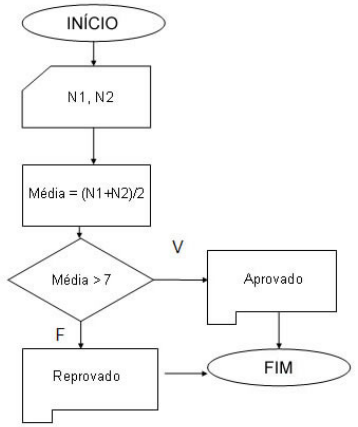






PSEUDOCÓDIGO

Algoritmo

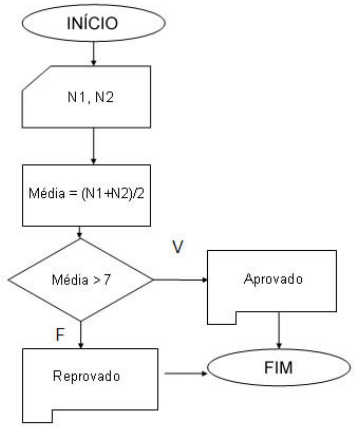


Forma de apresentar o algoritmo

```
Algoritmo1(n)
  para i ← 1 até n
      escreva("Hoje é dia de aula")
```

PSEUDOCÓDIGO

Algoritmo



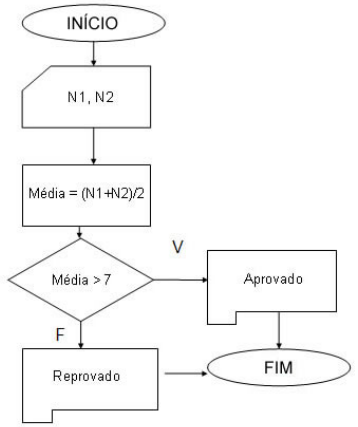
Forma de apresentar o algoritmo

Identificação

```
Algoritmo1(n)
  para i ← 1 até n
     escreva("Hoje é dia de aula")
```

PSEUDOCÓDIGO

Algoritmo



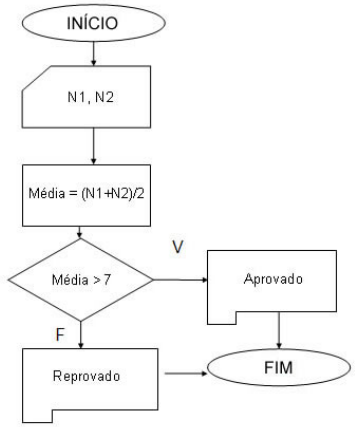
Forma de apresentar o algoritmo

Parâmetro de entrada

```
Algoritmo1(n)
  para i ← 1 até n
     escreva("Hoje é dia de aula")
```

PSEUDOCÓDIGO

Algoritmo



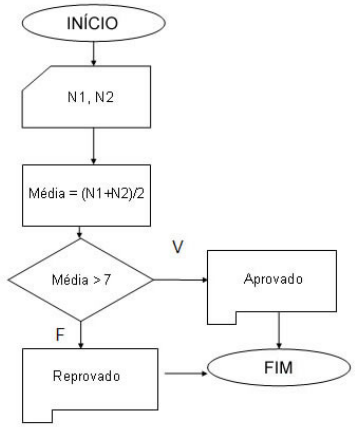
Forma de apresentar o algoritmo

```
Algoritmo1(n)
  para i ← 1 até n
      escreva("Hoje é dia de aula")
```

Comandos

PSEUDOCÓDIGO

Algoritmo



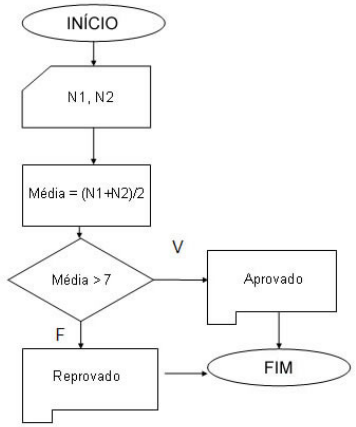
Forma de apresentar o algoritmo

```
Algoritmo1(n)
  para i ← 1 até n
    escreva("Hoje é dia de aula")
```

saída

PSEUDOCÓDIGO

Algoritmo



Pseudocódigo usado nos livros da bibliografia da disciplina

PROBLEMA

- **O enunciado de um problema especifica em termos gerais o relacionamento entre a entrada e a saída.**

- **Ex: Soma de 2 números inteiros**

Entrada: Dois números inteiros: *a*,*b*

Saída: Um número inteiro: *c*

Algoritmo2(*a*,*b*)
$c \leftarrow a + b$
retorne(*c*)

PROBLEMA

- **Ex: Busca em lista não ordenada**

Entrada: Uma sequência de *n* números $<a_1, ..., a_n>$ e uma chave de busca *x*.

Saída: O índice *i* da sequência onde a chave x se localiza (a_i = x) ou a indicação que a chave não se encontra na lista

PROBLEMA

- **Ex: Problema de Ordenação**

Entrada: Uma sequência de n números $\langle a_1, ..., a_n \rangle$

Saída: Uma permutação (reordenação) da sequência entrada, tal que $a_1 \leq a_2 \leq ... a_{n-1} \leq a_n$

INSTÂNCIA

Ao se fixar os parâmetros de um problema elabora-se um caso concreto ou instância.

INSTÂNCIA

Ao se fixar os parâmetros de um problema elabora-se um caso concreto ou instância.

- **Ex: Soma de 2 números inteiros**

Entrada: Dois números inteiros: *a*,*b*

Saída: Um número inteiro: *c*

- **Instância**

Entrada: *a* = 8, *b* = 23

INSTÂNCIA

- **Ex: Busca em lista não ordenada**

Entrada: Uma sequência de *n* números <a_1, ..., a_n> e uma chave de busca *x*.

Saída: O índice *i* da sequência onde a chave x se localiza (a_i = x) ou a indicação que a chave não se encontra na lista

- **Instância**

Entrada: *n* = 10, <56, 12, 34, 8, 10, 4, 101, 37, 9, 41>, *x* = 53.

INSTÂNCIA

O algoritmo deve ser capaz de resolver qualquer instância do problema

RESOLVENDO O PROBLEMA

1. Compreender o problema
2. Decidir que *técnica* vai utilizar para desenvolver o algoritmo (exato, aproximado) para o problema.
3. Desenvolver o algoritmo
4. *Provar que o algoritmo está correto*
5. *Analisar o algoritmo*
6. Codificar o algoritmo

ALGORITMO

- Podem existir diversos algoritmos para resolver o mesmo problema.



Qual escolher?

TÉCNICAS DE DESENVOLVIMENTO

Divisão-e-conquista

Programação Dinâmica

Algoritmo Guloso

Backtracking

Branch-and-bound



PROVA QUE O ALGORITMO ESTÁ CORRETO

- Prova que o algoritmo produz a resposta correta para qualquer instância do problema.

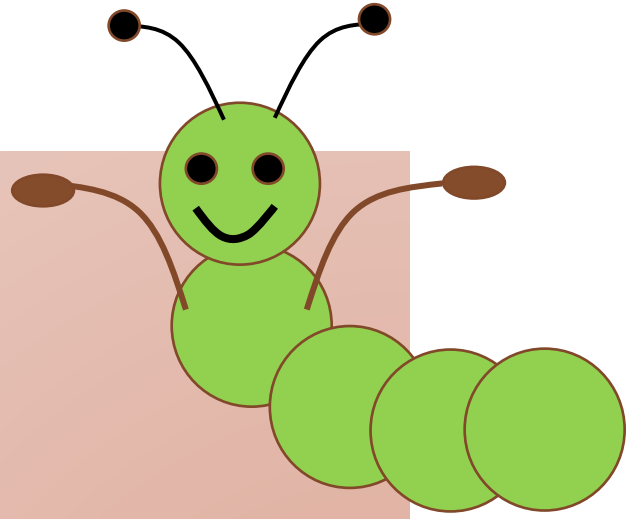
- Técnicas de prova:
  - Prova direta
  - Prova por contradição
  - Indução Matemática

ANÁLISE

- *Analisar* um algoritmo significa *predizer* os *recursos* que o algoritmo irá requerer quando executado
- *Eficiência*

Medidas mais comuns: tempo de processamento e quantidade de memória.

TEMPO DE PROCESSAMENTO



Que tal ?

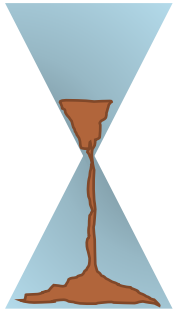
Implementar o algoritmo
Aplicar a várias instâncias do problema
Medir o tempo

Tempo de Processamento

O tempo de processamento pode não depender apenas do algoritmo implementado.

Depende também de:

- Tipo de computador
- Qualidade do compilador
- Habilidade do programador
- O algoritmo pode ter sido ajustado para ter bom desempenho naquele conjunto de instâncias.



Análise de Algoritmos

Dado um modelo abstrato de computador
(será visto adiante)



Quantificar os recursos computacionais necessários à solução de um problema (todos os casos) como função do tamanho das entradas das instâncias

Análise de Algoritmos

Como *recursos* entendemos a *memória*, o *tempo de processamento*, a *natureza das operações*, etc

O tamanho de uma instância corresponde ao *número de bits* necessários para *codificá-la* em um computador (*entrada*)

Tamanho da Entrada

Exemplo

Problema: Buscar o elemento x em uma lista com n inteiros.

Algoritmo:

```
Busca_lista_não_ordenada(L,x,n)
    i ← 1
    enquanto i ≤ n e L[i] ≠ x
        i ← i+1
    se i ≤ n retorne(i)
    senão retorne(-1)
```

Qual o tamanho da entrada?

Tamanho da Entrada



Exemplo: Considere uma lista de inteiros de tamanho *n*.
Algoritmos para *ordenação, busca, encontrar o menor elemento da lista*, etc...

Nestes casos o tamanho da entrada é o número de elementos da lista, ou seja, *n*.

Tamanho da Entrada

Exemplo

Problema: Soma de duas matrizes *n* x *n*.

Algoritmo:

```
Soma_matriz(A,B,n)
  para i ← 1 até n
      para j ← 1 até n
        C[i,j] ← A[i,j] + B[i,j]
  retorna(C)
```

Qual o tamanho da entrada?

Tamanho da Entrada

Exemplo: Matriz *n* x *n*.
Algoritmos para *soma, multiplicação*, etc...

O tamanho da entrada pode ser:

- a ordem da matriz, *n*.
- número de elementos na matriz, *N.*

  $N = n^2$ (matriz quadrada)

  $N = n \times m$ (matriz qualquer)

Tamanho da Entrada

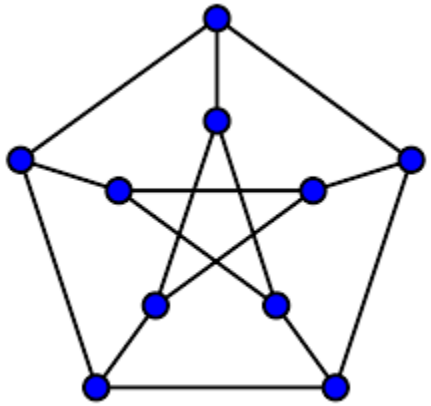
Como usamos representação binária, o tamanho da entrada, n, é medido pelo número de bits de representação, b.

$$b = \lfloor \log_2 n \rfloor + 1$$

Tamanho da Entrada

Alguns algoritmos requerem mais de um parâmetro para indicar o tamanho da entrada.

Por exemplo, o número de vértices e o número de arestas para algoritmos em grafos.



Estruturas de Dados

Meios para armazenar e organizar dados com o objetivo de facilitar o acesso e modificações.

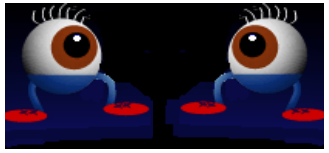


Cada estrutura possui pontos fortes e limitações.

Complexidade

A função complexidade deve independer dos recursos de hardware utilizados.

Porque?



Modelos

Existem diversos modelos de computação como:

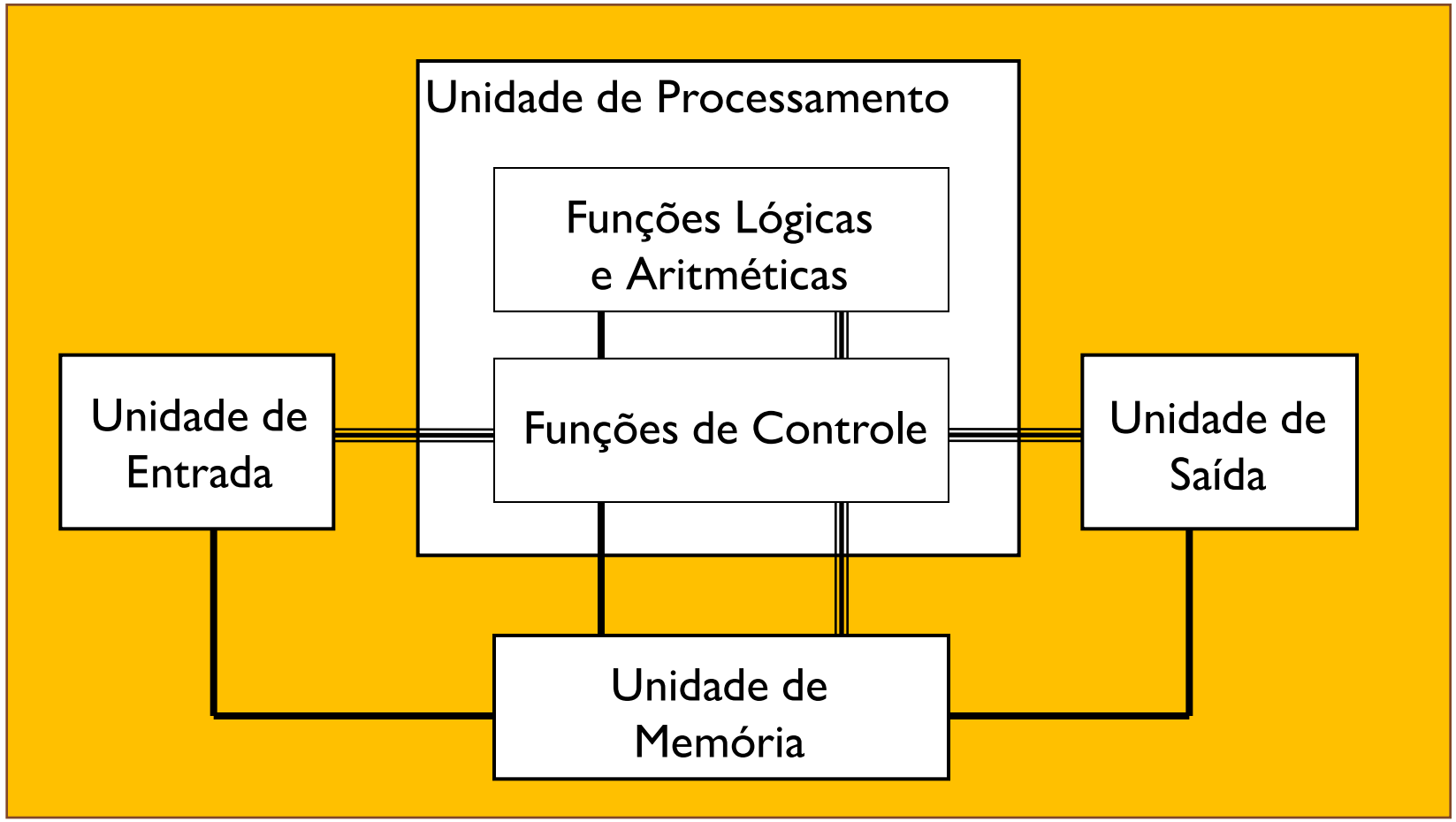
1. Máquina de Turing
2. O modelo RAM (Random Access Machine)
3. A Máquina Baseada em Ponteiros



Alan Mathison Turing

RAM possui um número infinito de registradores na memória, cada um podendo guardar um inteiro.

Máquina RAM


Unidade de Processamento
Funções Lógicas e Aritméticas
Funções de Controle
Unidade de Entrada
Unidade de Saída
Unidade de Memória

Máquina RAM

Operações

1. Operações de Entrada e Saída
2. Operações Aritméticas
3. Operações Lógicas e Relacionais
4. Movimentação de Dados entre os Componentes

Complexidade em Tempo

O modelo considera que *cada instrução I* possuirá um *tempo associado t(I)* para ser operacionalizada em RAM.

São processadas r_1 instruções do tipo I_1 , r_2 instruções do tipo I_2 , ..., r_m instruções do tipo I_m ,

Então o tempo total de execução *T(n)* será

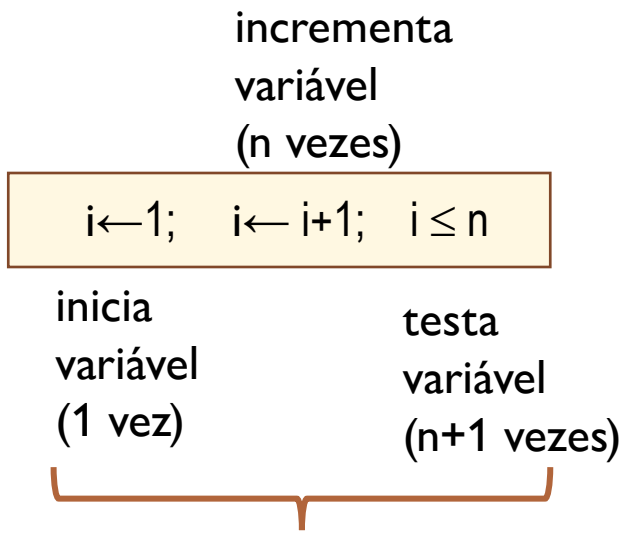
$$T(n) = \sum_{j=1}^{m} r_j t(I_j)$$

Tempo de Execução

Exemplo contagem de operações

Algoritmo:

```
Soma_matriz(A,B,n)
 para i ← 1 até n
     para j ← 1 até n
         C[i,j] ← A[i,j] + B[i,j]
 retorna(C)
```



total: 1 + n + n + 1 = 2n+2 operações

Tempo de Execução

Exemplo contagem de operações

Algoritmo:

```
Soma_matriz(A,B,n)
  para i ← 1 até n
      para i ← 1 até n
          C[i,j] ← A[i,j] + B[i,j]
  retorna(C)
```

O número de operações é uma função quadrática em *n*.

Complexidade em Tempo

t_A: $\{0,1\}^* \rightarrow \aleph$ é dita complexidade do algoritmo A se, para todo x, dado x como entrada de A, A termina em exatamente $t_A(x)$ passos.

Análises:

-Pior Caso

-Melhor Caso

-Caso Médio

Complexidade em Tempo



Pior Caso

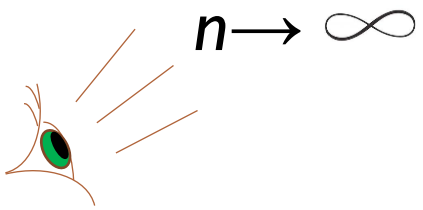
Estamos interessados na dependência da complexidade como função do tamanho da entrada, tomando o valor ***máximo*** de passos para entradas de tamanho relevante.

Sendo n o tamanho da entrada, temos $n \rightarrow \infty$.

Busca em lista linear não ordenada

Qual é a entrada que leva o algoritmo a fazer o maior número de operações?
Qual é a complexidade em tempo para esta entrada?

```
procedimento Busca Sequencial (L[1,..,n],x)
1.  i ← 1
2.  Enquanto i ≤ n e L[i] ≠ x
3.      i ← i+1
4.  Se i ≤ n retorne(i)
5.  Senão retorne(-1)
```

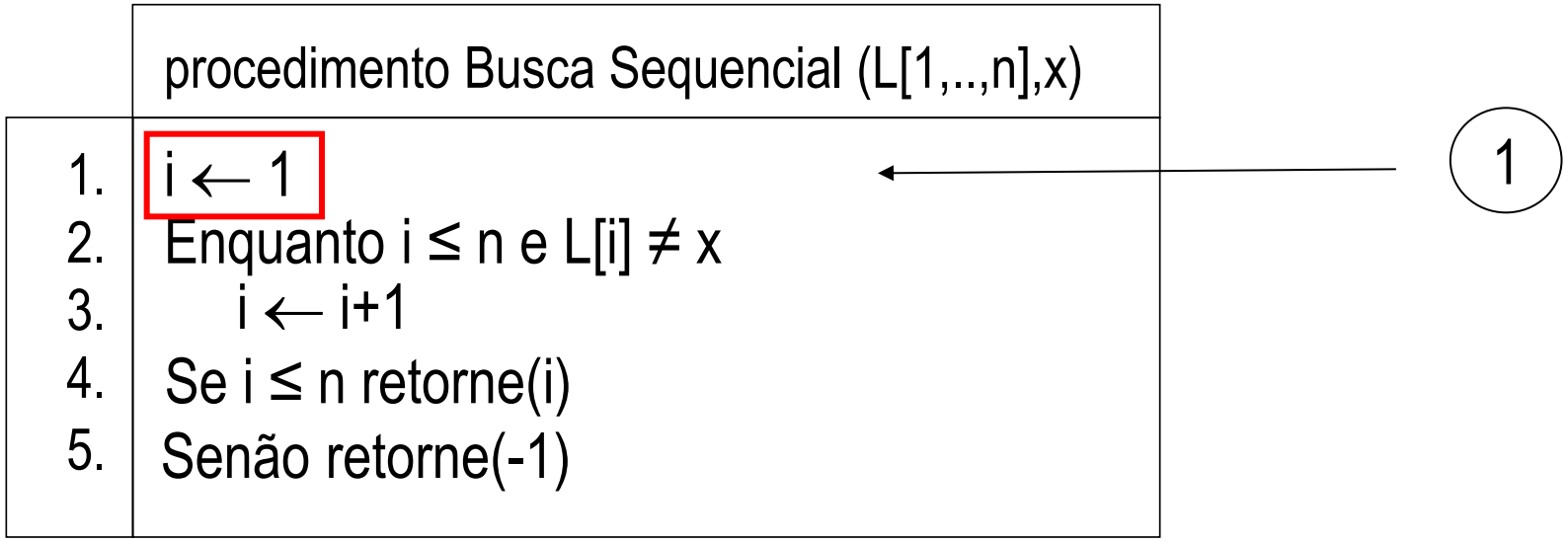


L =

150	50	10	103	20	23	345	57	5	653

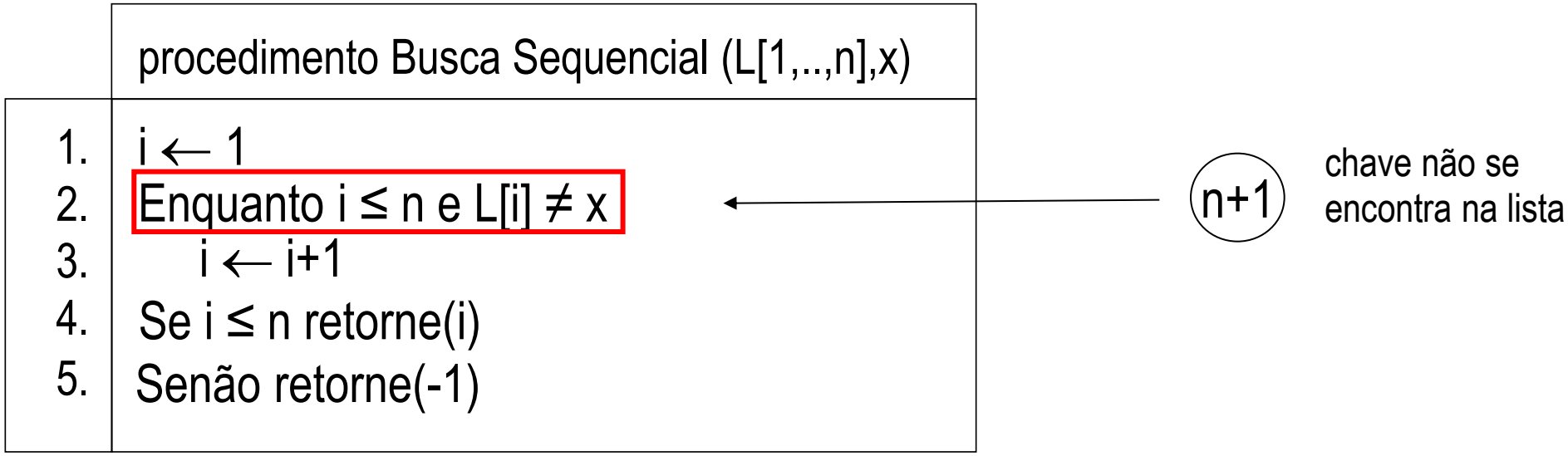
Exemplo

Pior Caso: A chave de busca se encontra na última posição da lista ou a chave não se encontra na lista.



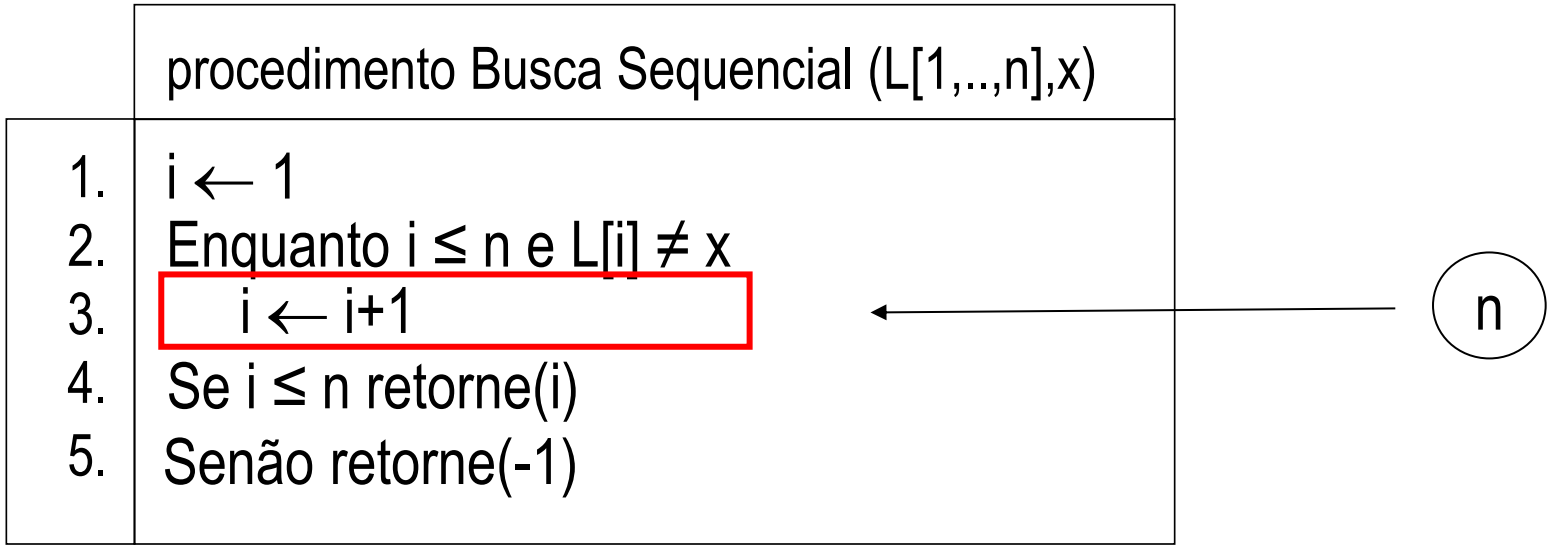
Exemplo

Pior Caso: A chave de busca se encontra na última posição da lista ou a chave não se encontra na lista.



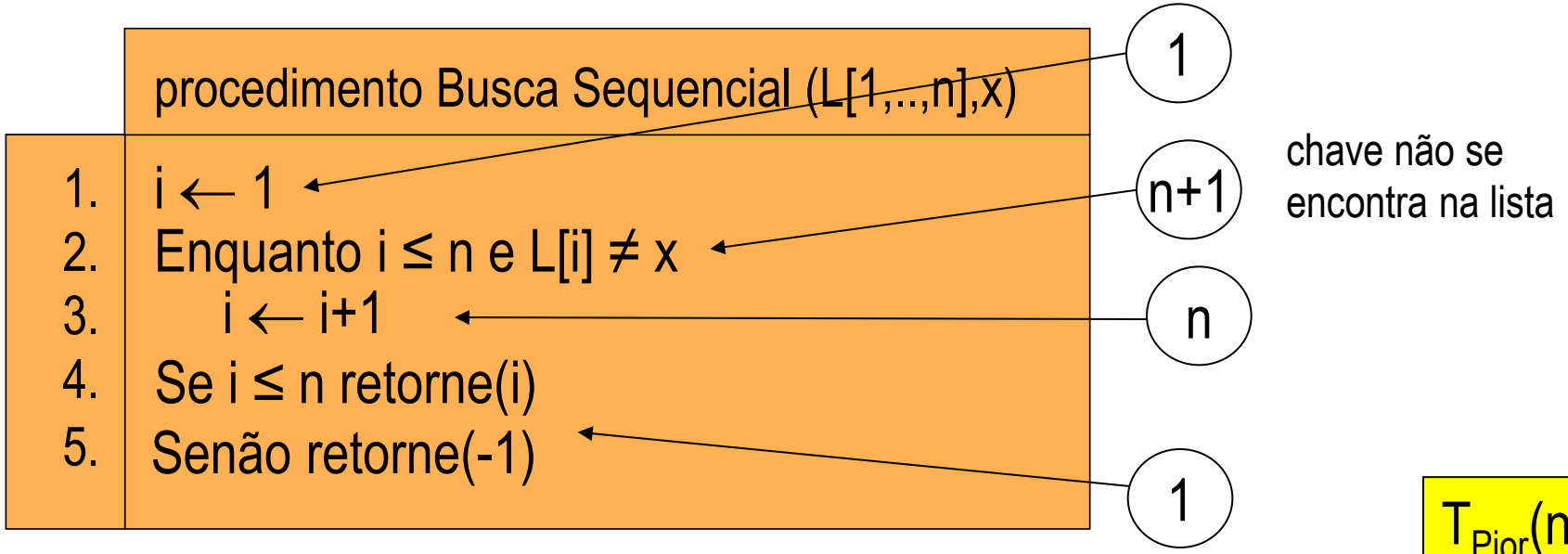
Exemplo

Pior Caso: A chave de busca se encontra na última posição da lista ou a chave não se encontra na lista.



Exemplo

Pior Caso: A chave de busca se encontra na última posição da lista ou a chave não se encontra na lista.



$T_{Pior}(n) = 2n+3$

Complexidade em Tempo

Melhor Caso



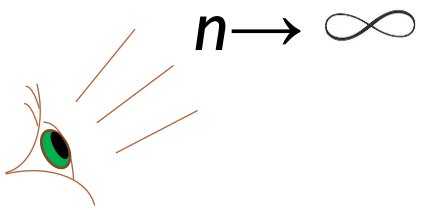
Estamos interessados na dependência da complexidade como função do tamanho da entrada, tomando o valor ***mínimo*** de passos para entradas de tamanho relevante.

Sendo n o tamanho da entrada, temos $n \rightarrow \infty$.

Exemplo

Qual é a entrada que leva o algoritmo a fazer o menor número de operações?
Qual é a complexidade em tempo para esta entrada?

```
procedimento Busca Sequencial (L[1,..,n],x)
1.  i ← 1
2.  Enquanto i ≤ n e L[i] ≠ x
3.      i ← i+1
4.  Se i ≤ n retorne(i)
5.  Senão retorne(-1)
```

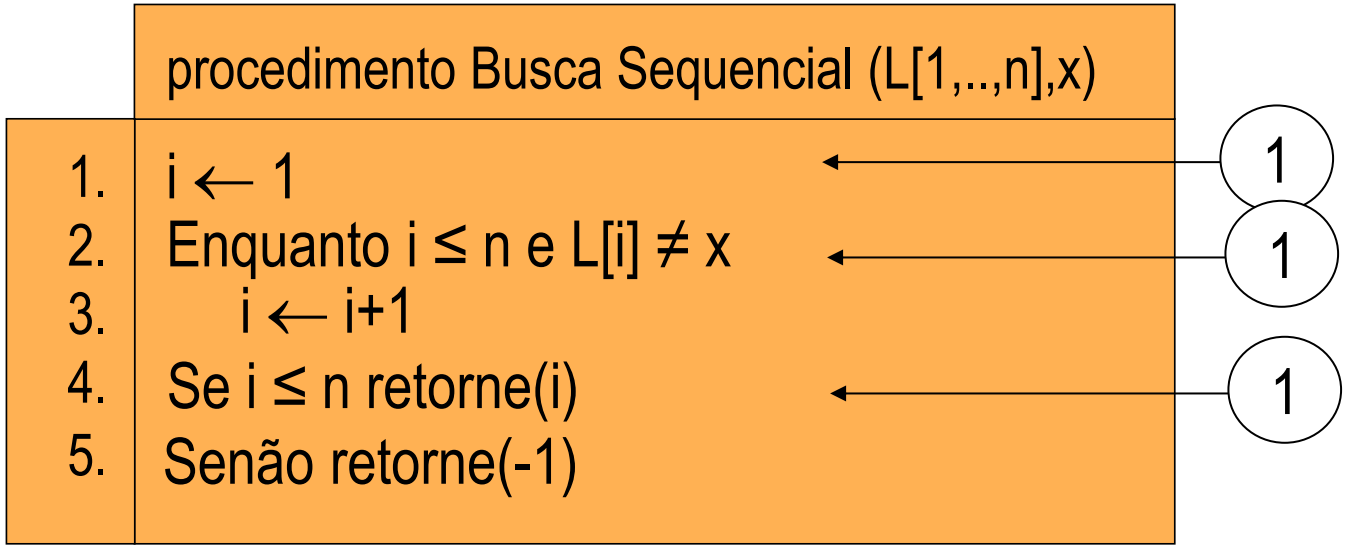


L =

150	50	10	103	20	23	345	57	5	653

Exemplo

Melhor Caso: A chave de busca se encontra na primeira posição da lista.



$T_{Melhor}(n) = 3$

Caso Médio

Depende de probabilidades associadas às possíveis entradas para o algoritmo.

Será visto em aula específica.

Busca em Lista Linear Ordenada

```
procedimento Busca SequencialOrdenada(L[1,..,n],x)
1.  i ← 1
2.  Enquanto i ≤ n e L[i] < x
3.      i ← i+1
4.  Se i ≤ n e L[i] = x retorne(i)
5.  Senão retorne(-1)
```

Melhor Caso?
Pior Caso?

15	50	100	103	200	230	345	357	465	653

Ordenação por Inserção

```
1.  i ← 2
2.  faça
3.      x ← A[i]
4.      j ← i-1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ←j–1
9.      faça
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

procedimento inserção (A[1..*n*])

Ordenação por Inserção

```
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j – 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

procedimento inserção (A[1.*n*])

8	7	1	2	3

i=2

X=A[2] →X=7

j=1

7<A[1]

A[2]=A[1]=8

j=0

A[0+1]=7

i=3

7	8	1	2	3

Ordenação por Inserção

```
    procedimento inserção (A[1.n])
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j– 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

7	8	1	2	3

i=3

X=1

j=2

1<A[2]

A[3]=A[2]=8

j=1

7	8	8	2	3

Ordenação por Inserção

```
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j– 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

procedimento inserção (A[1.*n*])

7	7	8	2	3

j=1>0 x=1<7

A[2]=A[1]=7

j=0 ⟶

A[1]=1

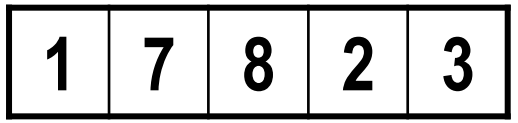
i=4

1	7	8	2	3

Ordenação por Inserção

```
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j– 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

procedimento inserção (A[1.*n*])



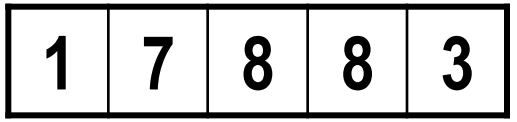
X=2

j=3

2<A[3]

A[4]=A[3]=8

J=2



Ordenação por Inserção

```
    procedimento inserção (A[1.n])
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j– 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

1	7	7	8	3

2<A[2]

A[3]=A[2]=7

J=1 2>A[1] ⟶

A[2]=2

i=5

1	2	7	8	3

Ordenação por Inserção

```
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j– 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

procedimento inserção (A[1.*n*])

1	2	7	8	3

i=5

X=3

j=4

J>0 3<A[4]

A[5]=A[4]=8

J=3

1	2	7	8	8

Ordenação por Inserção

```
    procedimento inserção (A[1.n])
1.  i = 2
2.  faça
3.      x ← A[i]
4.      j ← i- 1
5.      enquanto ((j > 0) e (x < A[j]))
6.
7.          A[j+1] ← A[j]
8.          j ← j– 1
9.      fim
10.     A[j+1] ← x
11.     i ← i + 1
12. enquanto (i <=n)
```

1	2	7	7	8

J>0 3<A[3]

A[4]=A[3]=7

J=2 3>A[2] ⟶

A[3]=3

i=6

1	2	3	7	8

Ordenação por Inserção

Prova que o algoritmo é correto

1. Para i = 2 a lista *A* = [1...*i*-1] está ordenada.

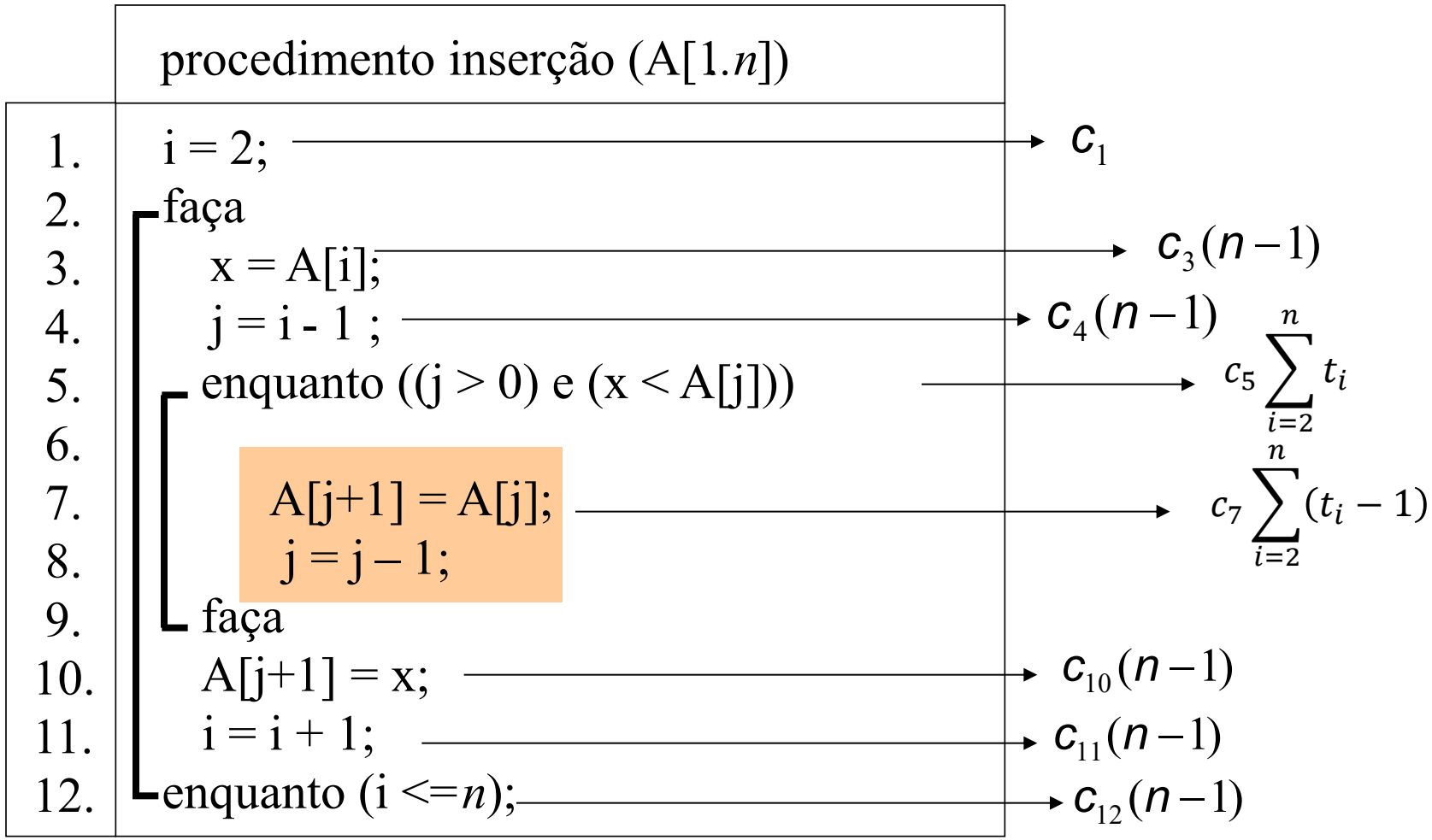
2. O laço externo funciona deslocando os elementos *A*[*i*-1], *A*[*i*-2],..., uma posição à direita, até encontrar a posição adequada para *A*[*i*] onde o elemento é inserido. Portanto, após uma iteração do laço externo, a lista com *i* elementos está ordenada.

Ordenação por Inserção

Prova que o algoritmo é correto

3. O laço externo termina quando *i* é maior que o número de elementos em *A*, denotado por *n*. Portanto, a lista com *n* elementos, ao final da iteração do laço externo está ordenada.

Análise da Ordenação por Inserção



Melhor Caso

```
                procedimento inserção (A[1.n])
1.    i = 2;                                          → c1
2.    faça
3.        x = A[i];                                   → c3 (n – 1)
4.        j = i - 1 ;                                 → c4 (n – 1)
5.        enquanto ((j > 0) e (x < A[j])) faça        → c5 (n – 1)
6.        {
7.            A[j+1] = A[j];
8.            j = j – 1;
9.        }
10.       A[j+1] = x;                                 → c10 (n – 1)
11.       i = i + 1;                                  → c11 (n – 1)
12.   enquanto (i <=n);                               → c12 (n – 1)
```

Lista de entrada está ordenada

Pior Caso

```
    procedimento inserção (A[1.n])
1.  i = 2;
2.  faça
3.     x = A[i];
4.     j = i - 1 ;
5.     enquanto ((j > 0) e (x < A[j])) faça
6.     {
7.        A[j+1] = A[j];
8.        j = j – 1;
9.     }
10.    A[j+1] = x;
11.    i = i + 1;
12. enquanto (i <=n);
```

c_1

$c_3(n-1)$

$c_4(n-1)$

$c_5(\frac{n(n+1)}{2}-1)$

$c_7(\frac{n(n-1)}{2})$

$c_{10}(n-1)$

$c_{11}(n-1)$

$c_{12}(n-1)$

Lista de entrada está na ordem inversa

Cálculos

Como os cálculos foram feitos ?

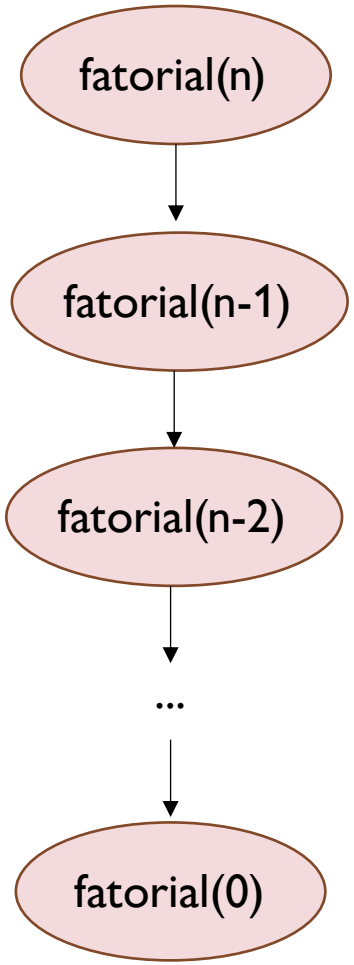
$$\sum_{i=2}^{n} i = \frac{n(n+1)}{2} - 1 \longrightarrow c_5\left(\frac{n(n+1)}{2} - 1\right)$$

$$\sum_{i=2}^{n} (i-1) = \frac{n(n-1)}{2} \longrightarrow c_7\left(\frac{n(n-1)}{2}\right)$$

Algoritmos Recursivos

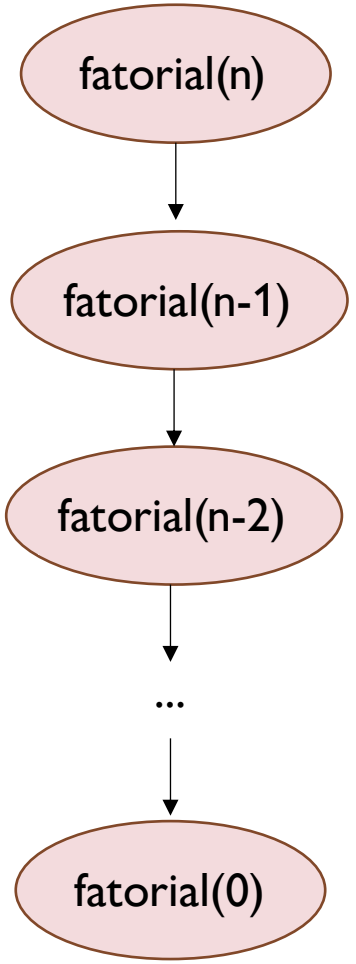
Cálculo do Fatorial de *n*

```
fatorial (n)
    se n == 0 então
        retorne(1)
    senão
        retorne(n * fatorial (n-1) )
```



Cálculo do Fatorial de *n*

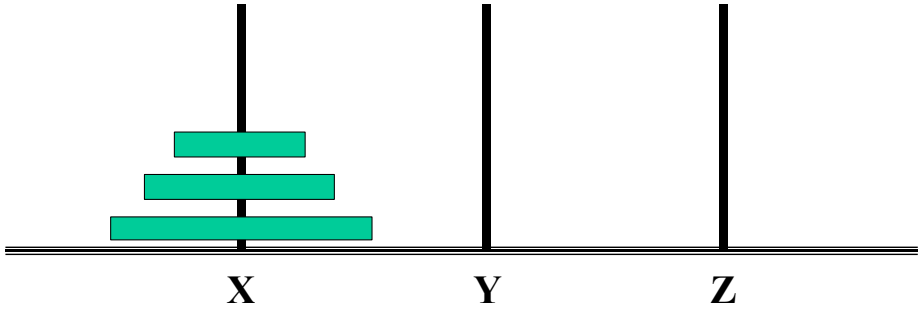
```
fatorial (n)
    se n == 0 então
        retorne(1)
    senão
        retorne(n * fatorial (n-1) )
```



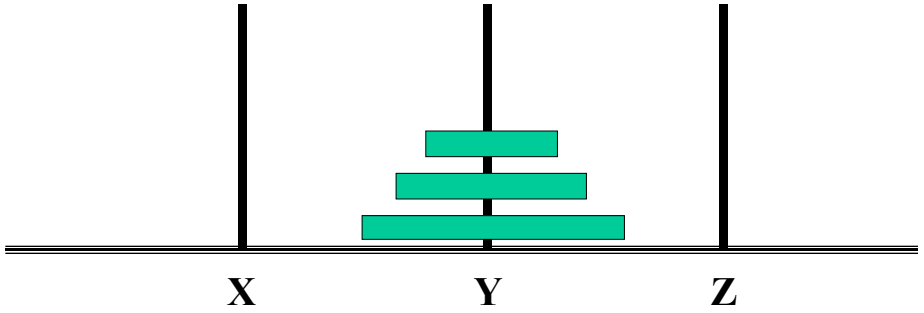
$$T(n) = \begin{cases} t_1 & para\ n = 0 \\ T(n-1) + t_2 & para\ n > 0 \end{cases}$$

Torres de Hanoi

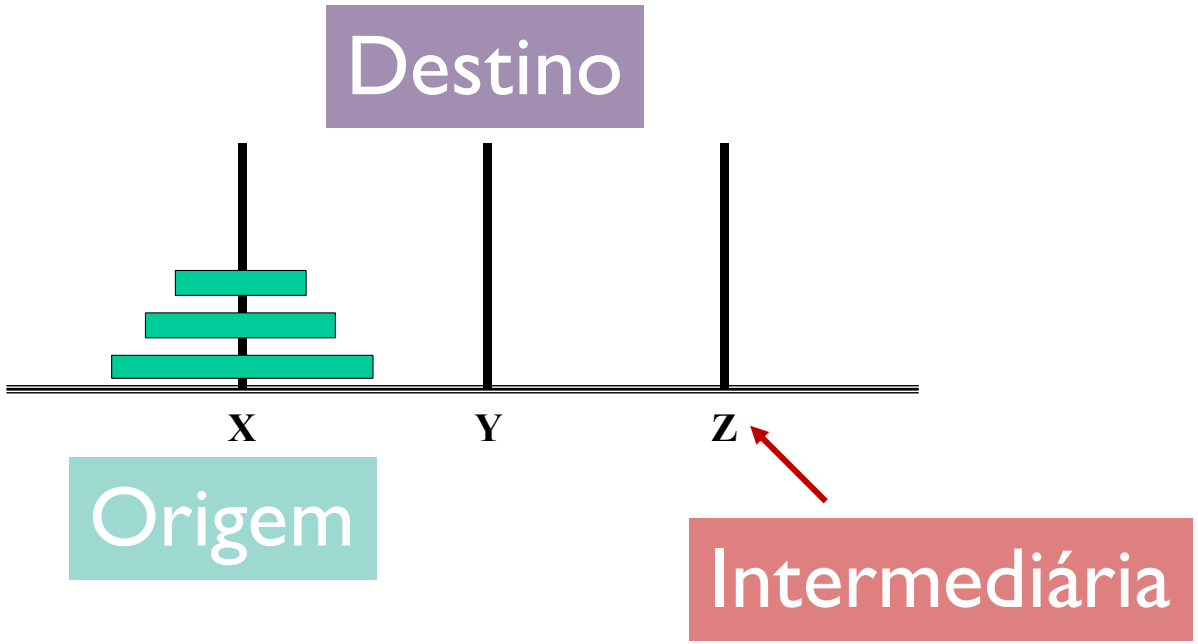
Entrada



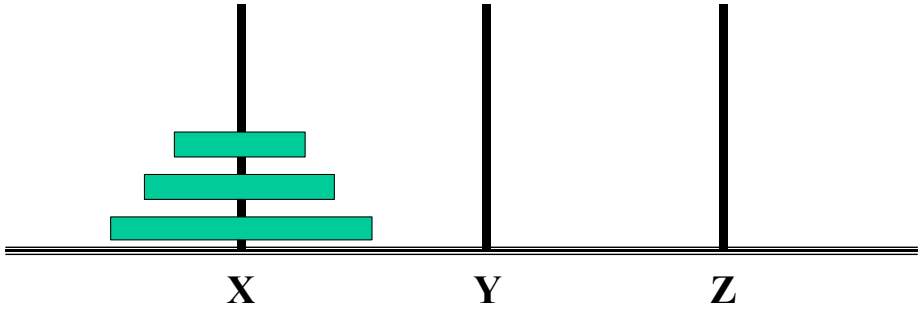
Saída

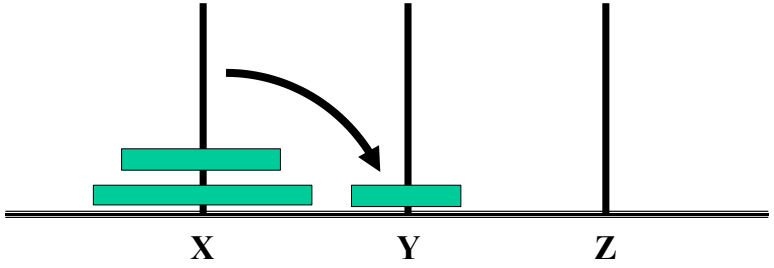


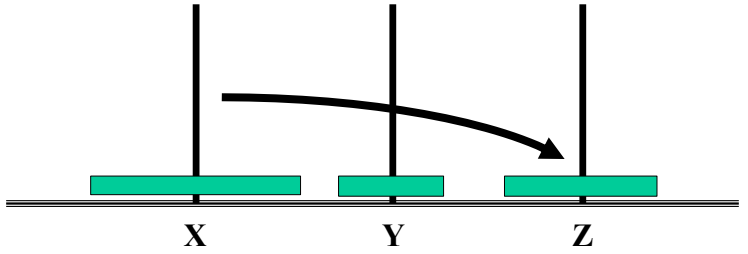
Torres de Hanoi



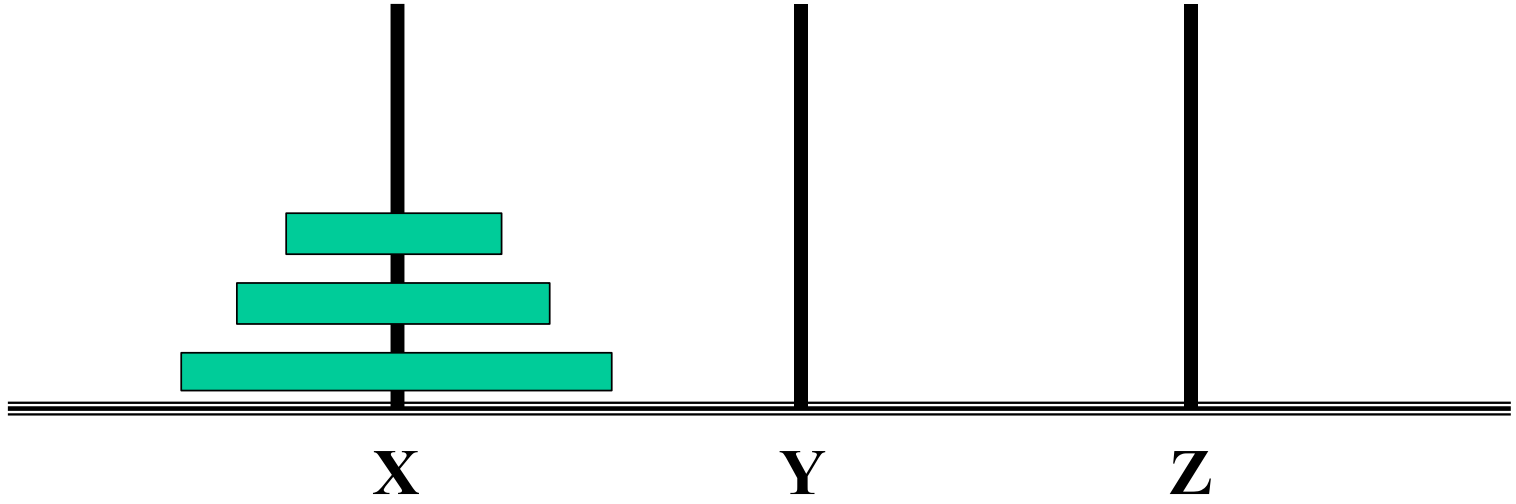
Torres de Hanoi

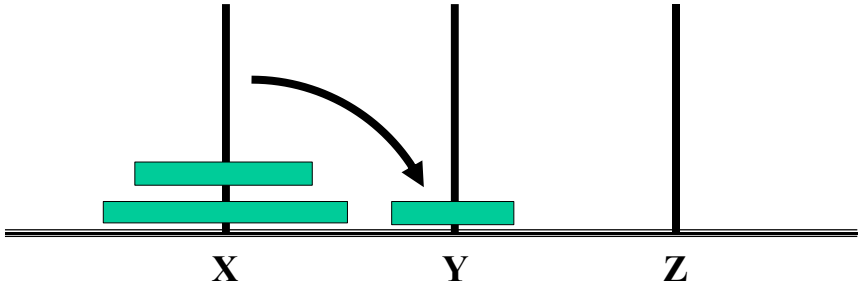


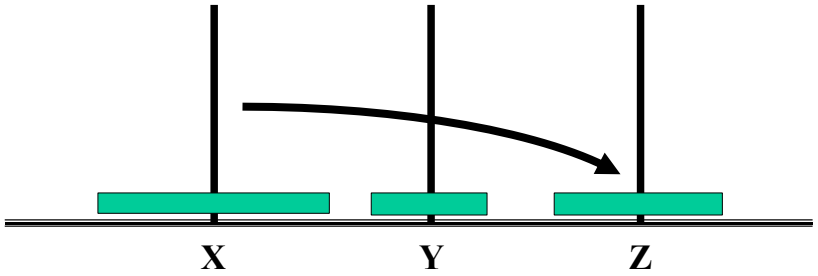




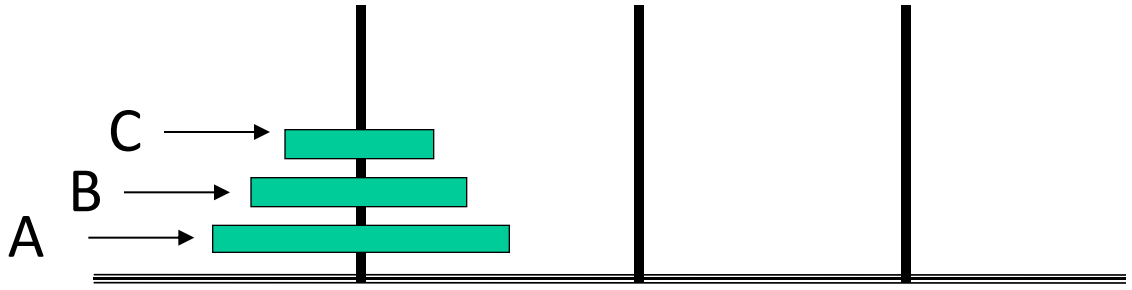
Torres de Hanoi







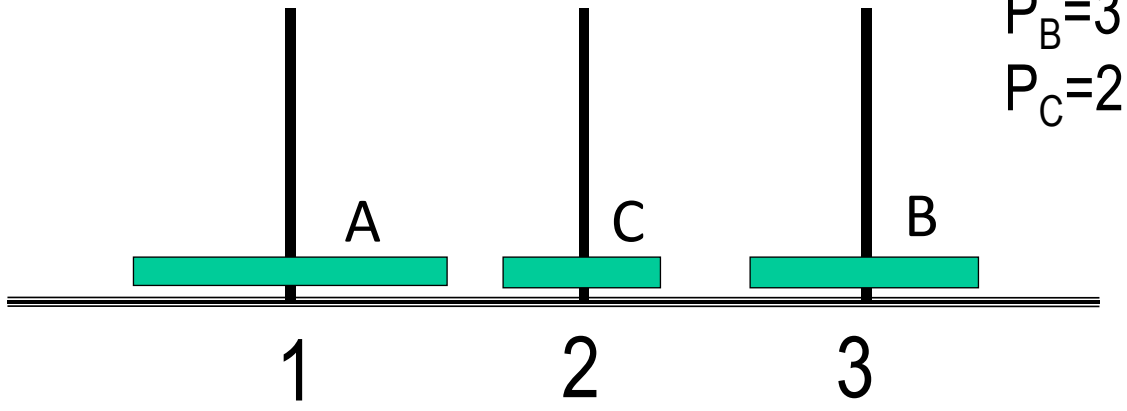
Torres de Hanoi



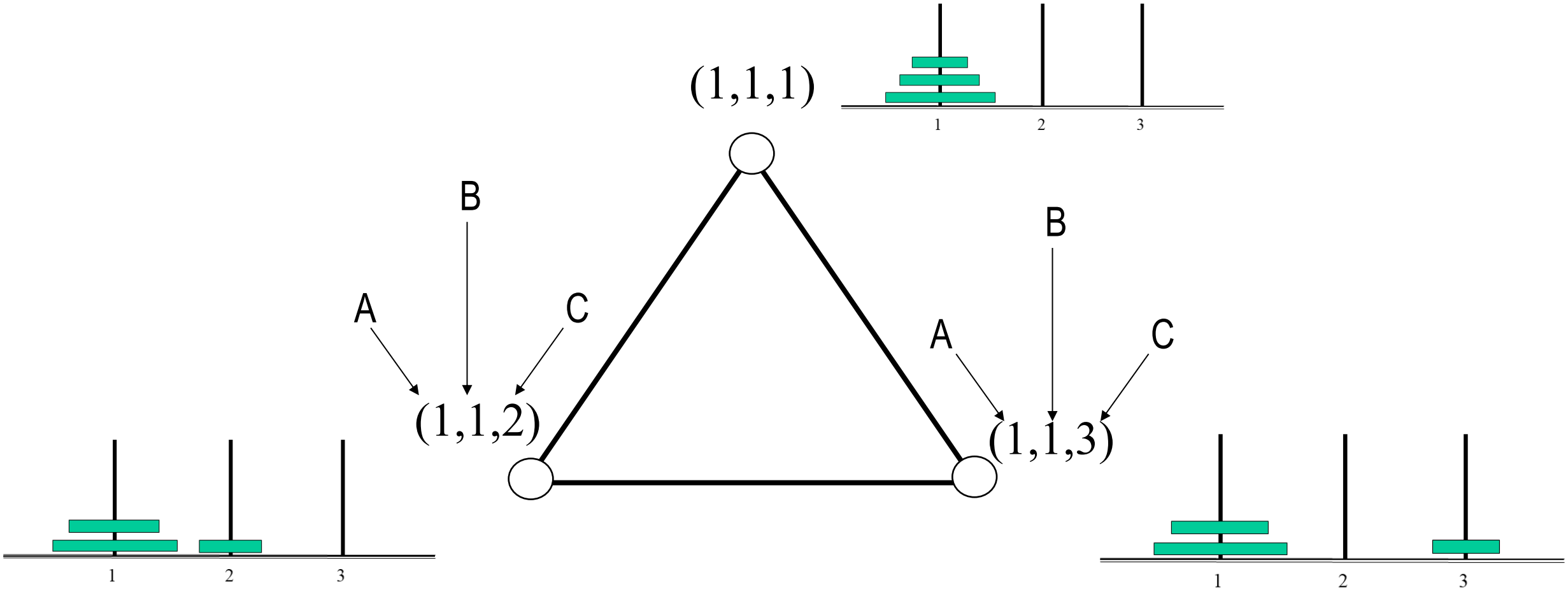
(1,1,1)

(1,3,2)

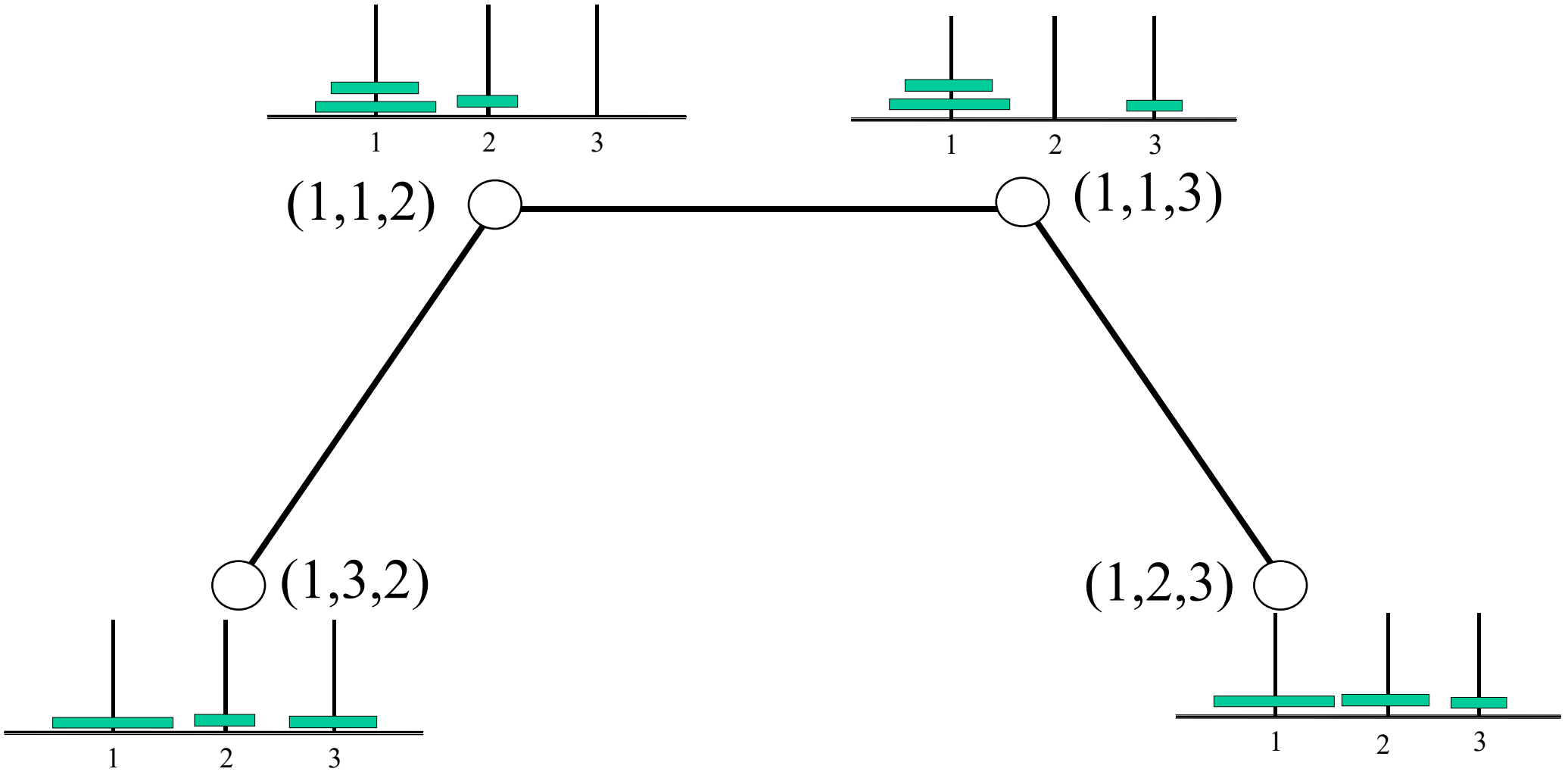
P_A=1
P_B=3
P_C=2



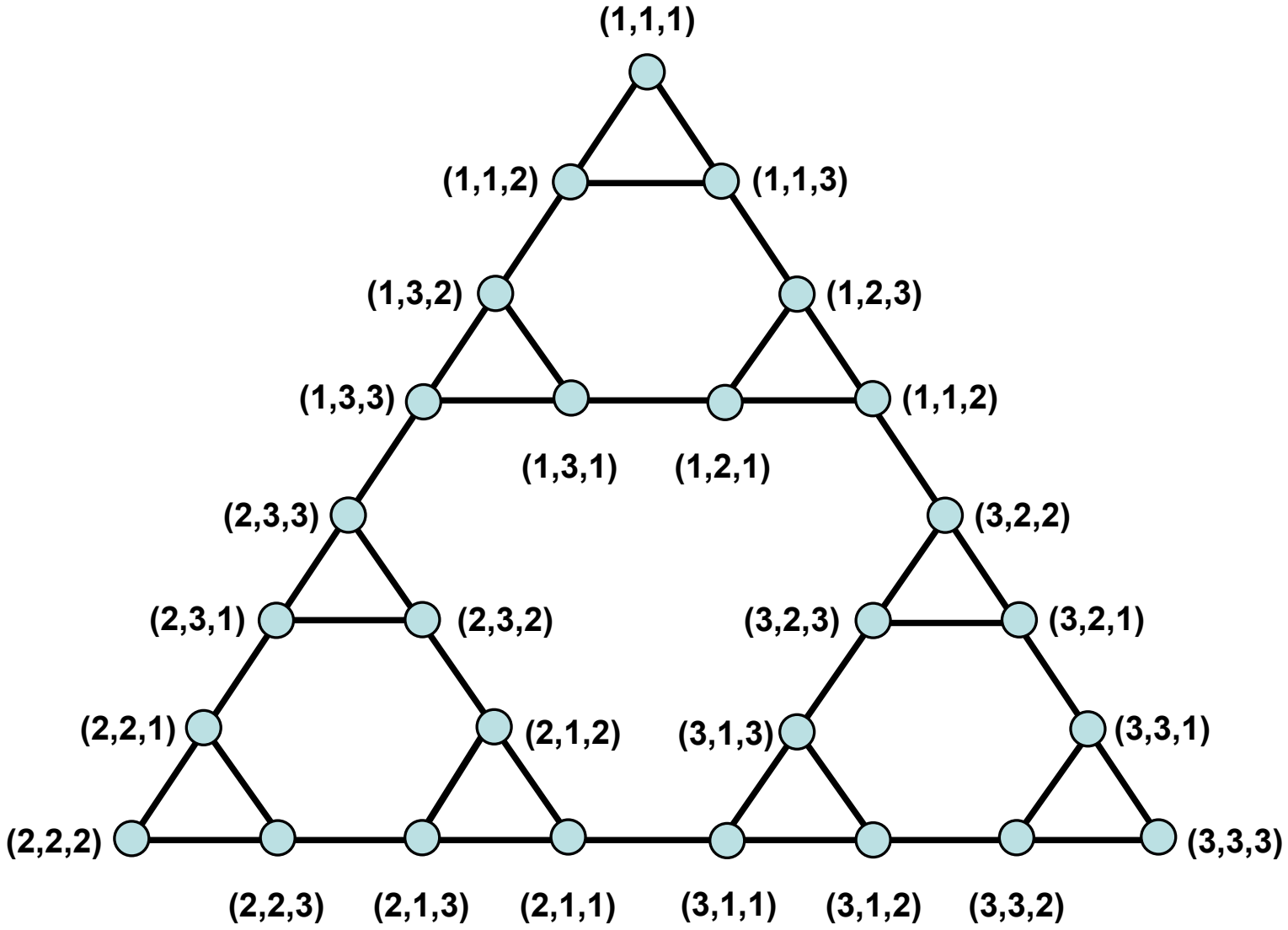
Torres de Hanoi



Torres de Hanoi

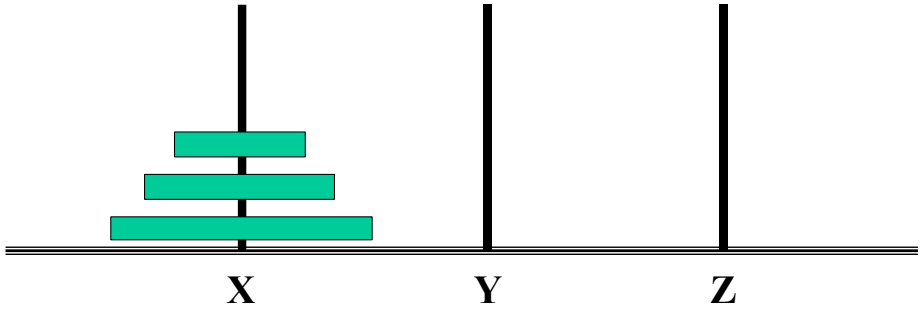


Torres de Hanoi



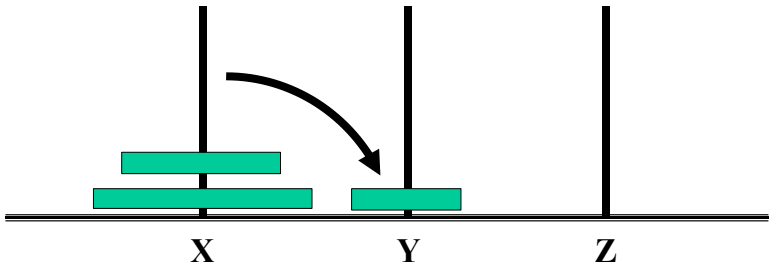
3^3 *Configurações*

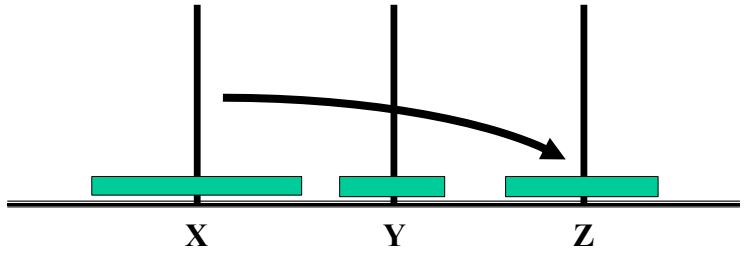
Torres de Hanoi



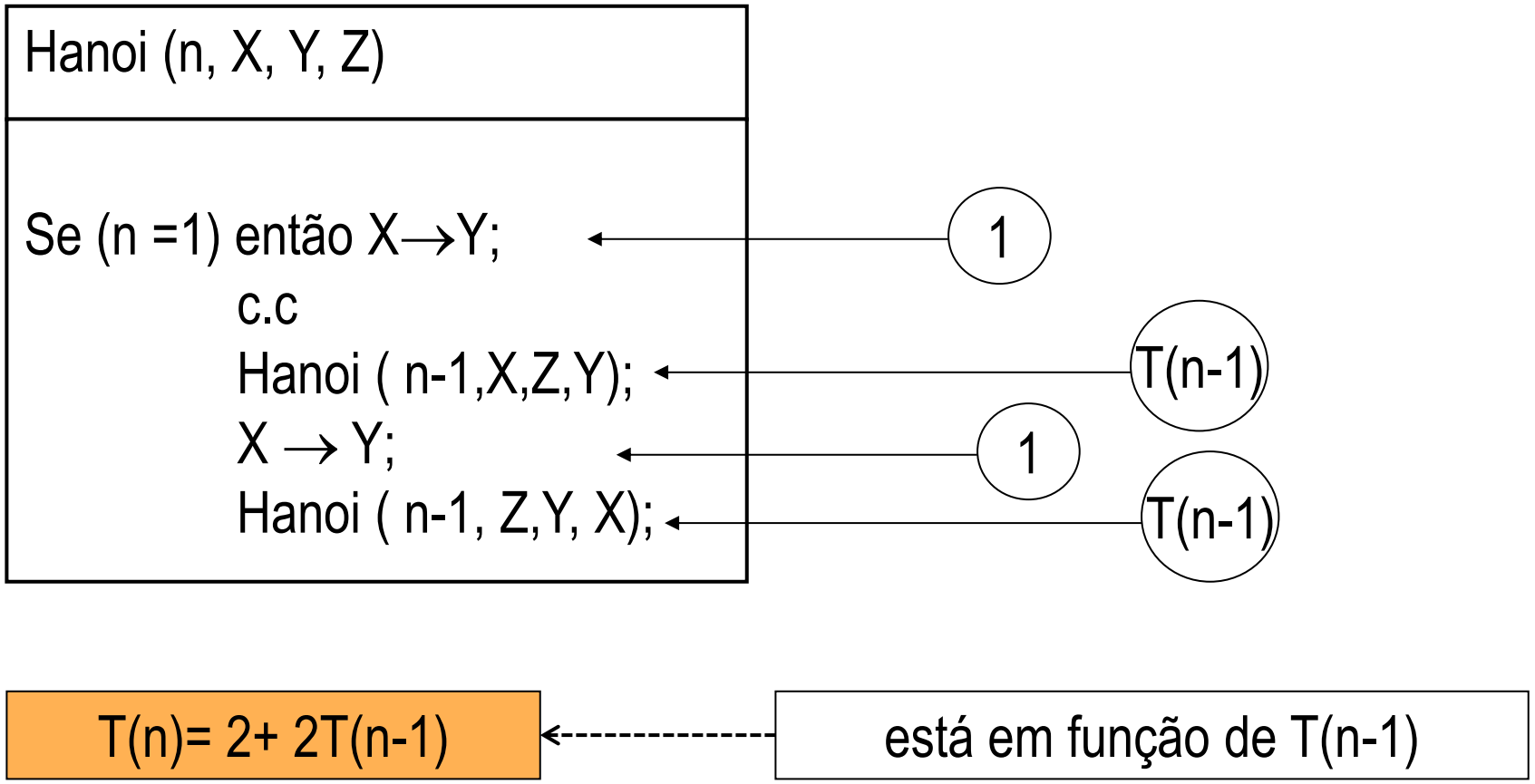

```
Hanoi (n, X, Y, Z)

Se (n =1) então X→Y;
Senão
    Hanoi ( n-1,X,Z,Y);
    X → Y;
    Hanoi ( n-1, Z,Y, X);
```





Algoritmo Recursivo



T(n)= 2+ 2T(n-1) ← está em função de T(n-1)

Complexidade em Espaço

Quantidade de memória consumida na computação para guardar resultados intermediários, os quais são armazenados nas células de trabalho (memória).

e_A: $\{0,1\}^* \rightarrow \aleph$ é dita complexidade do algoritmo A se, para todo x, dado x como entrada de A, A acessa exatamente $e_A(x)$ células de trabalho durante a execução de A.